

Operations Research

Linear programming I: Modeling with linear programming

Prof. Dr. Martin Bichler

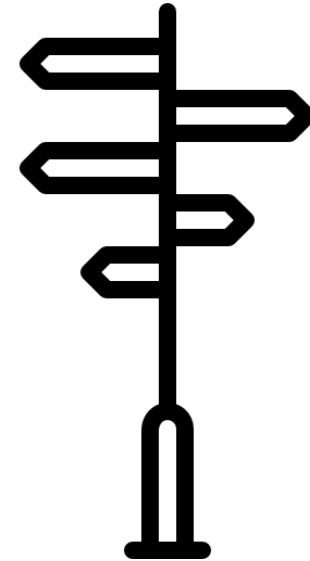
Department of Computer Science

School of Computation, Information, and Technology

Technical University of Munich

Course outline

- **Linear Optimization**
 - **Modeling with linear programming and graphical solution**
 - Solving linear programs, convexity
 - Simplex algorithm, 2-phase Simplex
 - Simplex algorithm in matrix notation, goal programming
 - Sensitivity analysis, reoptimization
 - Duality theory, zero-sum games
- **Integer Linear Optimization**
 - Modeling integer programming problems (IPs)
 - Solving IPs: branch & bound, cutting planes
 - Advanced solution methods: column generation, approximation algorithms
 - Fast solvable integer problems: total unimodularity, matroids
 - Network flow problems
 - Traveling salesperson problem
- **Nonlinear Optimization**
 - Optimization of multidimensional functions
 - Convex optimization



Today you will learn ...

- To understand **Linear Programming (LP)** as a tool in optimization.
- To define **characteristics of LPs** and to compare them with other modeling options.
- To **model** optimization problems as LP.
- To **solve** small LPs **graphically**.
- To model LPs and to have them solved with **Gurobi** or **Google OR Tools**.



=> Not all examples will be on slides today.

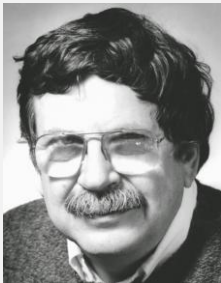
Optimization models

- Optimization problems
 - Maximization/minimization of **objective functions** (often under **restrictions/constraints**)
- a corresponding mathematical model is used to solve
 - the model captures the essential features of the problem in the form of **parameters** (constants) and **variables**
- an optimization model contains variables, an objective function and constraints
 - **a model instance replaces the model parameters with explicit numbers**

Linear Programming applications

Eugene Lawler (1980):

[Linear programming] is used to allocate resources, plan production, schedule workers, plan investment portfolios and formulate marketing (and military) strategies. The versatility and economic impact of linear programming in today's industrial world is truly awesome.



Eugene Lawler was an US-American mathematician, electrical engineer and computer scientist, who is one of the founders of combinatorial optimization.

Ressource: de.wikipedia.org

Linear Programming applications

Planning scarce resources is a central managerial task.

LP is important in numerous applications today:

- Design a production planning
- Transportation planning
- Allocation problems
- Mixing problems
- Multi-period rostering
- ...

Production planning

- **Objective:**
Select a set of products and services that maximize profit in a period.
- **Decision variables:**
How much of each product should be produced and sold in the planning period?
- **Constraints:**
 - Maximum quantity of each product
 - Minimum quantity to be produced per product
 - Maximum quantity of available resources

Example production planning

ACME introduces two new bike frames (Deluxe and Professional), made of aluminum and steel.

The profit per piece is 10 € for Deluxe and 15 € for Professional.

The table shows the proportions of aluminum and steel for each bike frame.

	<u>aluminum</u>	<u>steel</u>
Deluxe	2	3
Professional	4	2

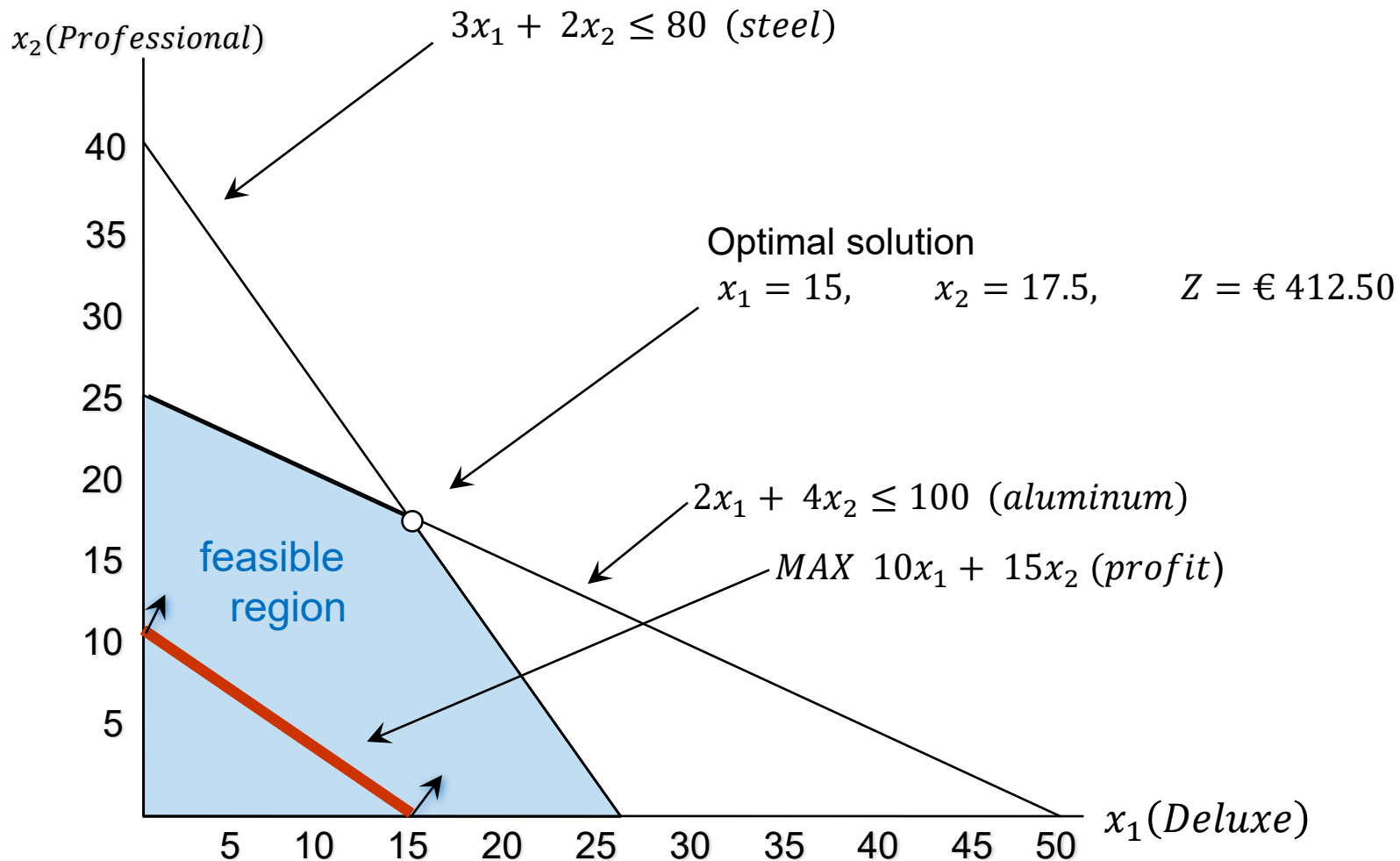
A supplier delivers 100 kg of aluminum and 80 kg of steel per week.

How many Deluxe and Professional bike frames can ACME produce per week?

Graphical solution (for two variables)

- Create a coordinate system in the two dimensions.
- Draw the restrictions as straight lines as if they were equations.
- Specify the feasible region.
- Draw the objective function as a straight line and shift the objective function in the direction of x_1 and x_2 in the objective function (at max!) so that it still touches the feasible region.
- The solution is the point where the objective function touches the feasible region.

Graphical solution



Definitions

Linear Program:

$\max \quad 10x_1 + 15x_2$					Objective function
s.t.	$2x_1 + 4x_2 \leq$	100	Restrictions / Constraints	Right-hand side	
	$3x_1 + 2x_2 \leq$	80			
	$x_1, x_2 \geq$	0	(Decision) Variables, Domains (lower / upper bounds)		

Standard software for optimization

Product	Producer	Website
COPT	Shanshu.ai	https://www.shanshu.ai/copt/
Gurobi Optimizer	Gurobi	www.gurobi.com
SoPlex	Zuse Institut	https://soplex.zib.de/
FICO Xpress	FICO	fico.com
LINDO	Lindo Systems	www.lindo.com
cuPDLP-C	Open Source	https://github.com/COPT-Public/cuPDLP-C
Lp_solve	Open Source	lpsolve.sourceforge.net/5.5/
COIN-OR	Open Source	http://www.coin-or.org/projects/
Google OR-Tools	Open Source	https://developers.google.com/optimization/



Get the License That's Right for You

We make it easy for students, faculty, and researchers to work with mathematical optimization. Whether for use in class or research, academics can **use Gurobi Optimizer at no cost**. Get all the same Gurobi features and performance, with no limits on model size.

Request the academic license that's right for you. Need help choosing? Please check out the options below.



Academic WLS License

Run Gurobi in virtually all deployment scenarios, including containerized environments, virtual machines, and bare-metal machines, across Linux, macOS, and Windows.

[Learn More](#)



Academic Named-User License

Start your journey here. Get a free, unlimited-use Gurobi Optimizer license for a single person, on a single machine.

[Learn More](#)



Academic Site License

University Administrators: You can share Gurobi with multiple users in your academic departments, schools, and classrooms by running Gurobi on your university's local-area network, at no cost to you or your institution.

[Learn More](#)



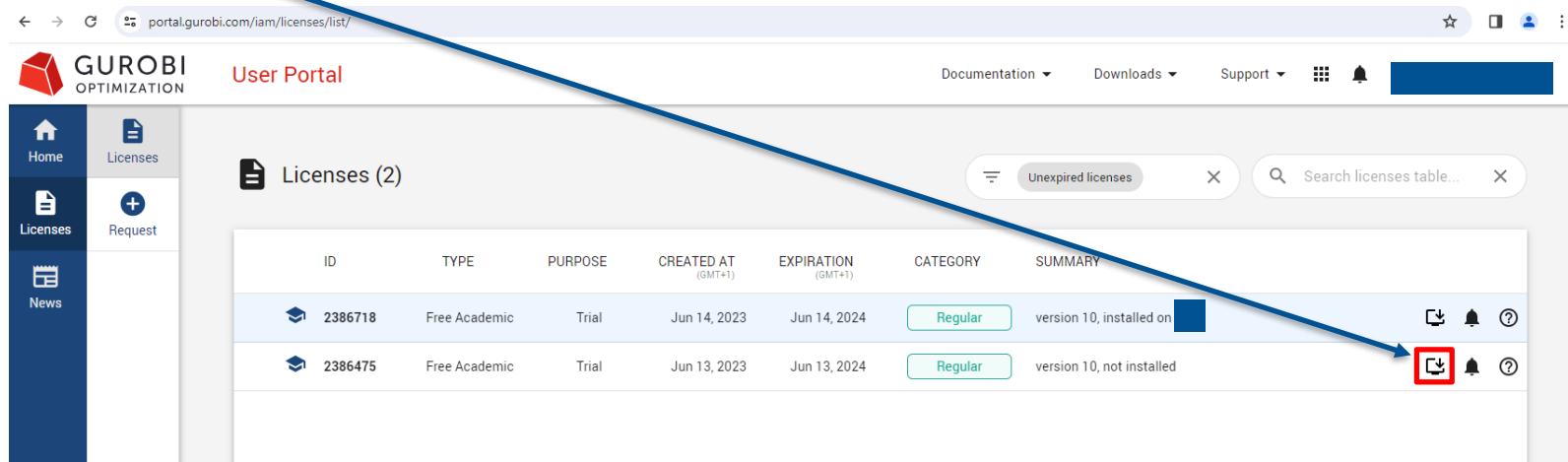
AMPL, GAMS, and MPL Programs

Are you an AMPL, GAMS, or MPL user? Learn about our academic license programs.

[Learn More](#)

How do I use Gurobi in a Python project?

1. Once you created an account and got an academic license, go to Licenses and follow the instructions next to the license you want to use:



The screenshot shows the Gurobi User Portal at portal.gurobi.com/iam/licenses/list/. The page displays a table of licenses. A blue arrow points from the text 'follow the instructions next to the license you want to use' to a red box around the download icon for license 2386475.

ID	TYPE	PURPOSE	CREATED AT (GMT+1)	EXPIRATION (GMT+1)	CATEGORY	SUMMARY	
2386718	Free Academic	Trial	Jun 14, 2023	Jun 14, 2024	Regular	version 10, installed on [blue box]	[download icon] [bell icon] [help icon]
2386475	Free Academic	Trial	Jun 13, 2023	Jun 13, 2024	Regular	version 10, not installed	[download icon] [bell icon] [help icon]

2. Install gurobipy: <https://support.gurobi.com/hc/en-us/articles/360044290292-How-do-I-install-Gurobi-for-Python->

Python

Install Python from python.org

Create a virtual environment

```
pip install -r requirements.txt
```

```
ipykernel
ipython
matplotlib
numpy
pandas
scikit-learn
scipy
notebook
jupyter
Statsmodels
```

```
pip list
```

Install solvers

```
python -m pip install gurobipy
```

or

```
python -m pip install ortools
```

Go through `python_tutorial.ipynb`

Learning the very basics in Python

Disclaimer

This is a very brief Python tutorial, where we assume that you already know the basics of Java. The tutorial is neither meant to be elaborate nor detailed, but rather to give you a quick and easy way of becoming comfortable with reading and writing Python code that we deem necessary for successfully completing OR. Also, it is not meant to discuss advantages and disadvantages of both languages, but rather highlight main differences and similarities, so that you can get an idea of what Python is about.

Python installation

Before proceeding, make sure you have Python installed on your computer (see, e.g., <https://realpython.com/installing-python/>).

Similarities

Before we showcase a few differences between Java and Python, it is important to note that in the greater landscape of *all* programming languages, Java and Python are actually rather similar. Among the main similarities:

Go through `budget_allocation.ipynb`

Case Study - Advertising Budget Allocation

In this case study, we will model a simple advertising budget allocation problem using linear programming and solve it with Gurobi. Before proceeding, make sure you:

- installed Python and `gurobipy`
- obtained an academic Gurobi license (by following the steps described in the lecture slides).

*To solve the small LP that we will formulate in this case study, you actually do not need a Gurobi license. However, a license is required to solve larger LPs.

Modeling in Gurobi and Python

Import Gurobi:

```
import gurobipy as gp  
from gurobipy import GRB
```

Initialize a model:

```
model = gp.Model('ModelName')
```

Add variables:

```
x = model.addVar(lower bound, upper bound, type, name)
```

Add constraints:

```
model.addConstr(...)
```


Modeling in Gurobi and Python

Define the objective function:

```
model.setObjective(... , sense)
```

Solve model:

```
model.optimize()
```

Official documentation:

<https://www.gurobi.com/documentation/11.0/refman/index.html>

Production program in Gurobi Python

```
import gurobipy as gp
from gurobipy import GRB

def create_production_plan():
    model = gp.Model('ProductionPlan')

    # Create variables
    x1 = model.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name='Deluxe')
    x2 = model.addVar(lb=0, ub=GRB.INFINITY, vtype=GRB.CONTINUOUS, name='Professional')

    # Add constraints
    model.addConstr(2 * x1 + 4 * x2 <= 100)
    model.addConstr(3 * x1 + 2 * x2 <= 80)

    model.setObjective(10 * x1 + 15 * x2, GRB.MAXIMIZE)

    # Optimize model
    model.optimize()
```

$$\begin{array}{ll} \text{Max} & 10x_1 + 15x_2 \\ \text{s.t.} & 2x_1 + 4x_2 \leq 100 \\ & 3x_1 + 2x_2 \leq 80 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \end{array}$$

Production program in Gurobi

```
if model.getAttr('Status') == GRB.OPTIMAL:  
    print(f'x1 = {x1.X}')
```

```
    print(f'x2 = {x2.X}')
```

```
    print(f'Objective = {model.getObjValue().getObjVal()}')
```

Output:

```
Optimal objective  4.125000000e+02  
x1 = 15.0  
x2 = 17.5  
Objective = 412.5
```

Gurobi

```
import gurobipy as gp
class productionplan:
    @staticmethod
    def main():
        try:
            env = gp.Env(empty=True)
            env.setParam("LogFile", "prod.log")
            env.start()
            model = gp.Model("prod", env)
            x1 = model.addVar(lb=0.0, ub=gp.GRB.INFINITY, obj=10.0,
                             vtype=gp.GRB.CONTINUOUS, name="Deluxe")
            x2 = model.addVar(lb=0.0, ub=gp.GRB.INFINITY, obj=15.0,
                             vtype=gp.GRB.CONTINUOUS, name="Professional")
            model.addConstr(2.0 * x1 + 4.0 * x2 <= 100.0, name="Stahl")
            model.addConstr(3.0 * x1 + 2.0 * x2 <= 80.0, name="Aluminium")
            model.setAttr(gp.GRB.Attr.ModelSense, gp.GRB.MAXIMIZE)
            model.optimize()
            print(f"{x1.VarName}: {x1.X}")
            print(f"{x2.VarName}: {x2.X}")
            print(f"Obj: {model.ObjVal}")
            model.dispose()
            env.dispose()
        except gp.GurobiError as e:
            print(f"Error code: {e.errno}. {e.message}")
if __name__ == "__main__":
    Produktionsplan.main()
```

Google OR-Tools



- Model and solve optimization problems in Python, C++, Java, or C#
- After modeling your problem in the programming language of your choice, you can use different solvers to solve it:
 - commercial solvers such as CPLEX, Gurobi or Fico Xpress, or
 - open-source solvers such as SCIP, GLPK or Google's GLOP and CP-SAT.
- **OR-Tools lets you keep the model and swap the backend** to HiGHS, SCIP, or Gurobi when needed.

Google OR-Tools

```
from ortools.linear_solver import pywraplp
def create_production_plan_or_tools():
    solver = pywraplp.Solver.CreateSolver('GLOP')
    if not solver:
        print("Solver not created.")
        return
    x1 = solver.NumVar(0, solver.infinity(), 'Deluxe')
    x2 = solver.NumVar(0, solver.infinity(), 'Professional')
    solver.Add(2 * x1 + 4 * x2 <= 100)
    solver.Add(3 * x1 + 2 * x2 <= 80)
    solver.Maximize(10 * x1 + 15 * x2)
    status = solver.Solve()
    if status == pywraplp.Solver.OPTIMAL:
        print(f'x1 = {x1.solution_value()}')
        print(f'x2 = {x2.solution_value()}')
        print(f'Objective = {solver.Objective().Value()}')
    else:
        print("The problem does not have an optimal solution.")

# Run the function
create_production_plan_or_tools()
```

General description of the optimal production plan

There are n products that can be produced.

There can be a profit of p_j per unit of product j .

Each product requires certain amounts of m resources:

- Each unit of a product j , with $j = 1, \dots, n$ consumes a_{ij} units of a resource i ,
with $i = 1, \dots, m$

A quantity u_i of resource i is available.

General production planning model

$$\begin{aligned} \max \quad & p_1x_1 + p_2x_2 + \cdots + p_nx_n \\ \text{s.t.} \quad & a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n \leq u_1 \\ & a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n \leq u_2 \\ & \quad \quad \quad \dots \\ & a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n \leq u_m \\ & x_j \geq 0 \text{ for } j = 1, \dots, n \end{aligned}$$

Or more compact:

$$\begin{aligned} \max \quad & \sum_{j=1}^n p_j x_j \\ \text{s.t.} \quad & \sum_{j=1}^n a_{ij} x_j \leq u_i \quad \text{for } i = 1, \dots, m \\ & x_j \geq 0 \quad \text{for } j = 1, \dots, n \end{aligned}$$

Google OR-Tools

No lower bound (i.e.,
the expression can be
as small as it wants)

For each product j its
coefficient in const. i
is $a[i][j]$

```
from ortools.linear_solver import pywraplp

# Create solver
solver = pywraplp.Solver.CreateSolver('GLOP') # GLOP = Google's LP solver

# Create variables
products = {}
for j in range(n):
    products[j] = solver.NumVar(0, solver.infinity(), f'Product{j}')

# Add constraints
for i in range(m):
    constraint = solver.Constraint(-solver.infinity(), rhs[i])
    for j in range(n):
        constraint.SetCoefficient(products[j], a[i][j])

# Set objective
objective = solver.Objective()
for j in range(n):
    objective.SetCoefficient(products[j], p[j])
objective.SetMaximization()

# Solve
status = solver.Solve()
```



The transportation problem

ACME has factories in Karlsruhe, Stuttgart and Cologne, which supply the company's warehouses in Budapest, Vienna and Munich.

The three warehouses require at least 4,000, 2,500 and 2,500 products per week.

The factories can each supply 3,000 bicycles per week.

Delivery costs:

Factories	Warehouses		
	<i>Budapest</i>	<i>Vienna</i>	<i>Munich</i>
<i>Karlsruhe</i>	12	6	5
<i>Stuttgart</i>	20	11	9
<i>Cologne</i>	30	26	28

Modeling of the transportation problem

- Variables:
 x_{ij} : Number of products to be delivered from i to j
- Objective function:
Minimization of the total delivery costs
- Constraints:
 - Demand in the warehouses
 - Supply in the factories

General transportation LP

$$\begin{aligned} \min \quad & \sum_i^n \sum_j^m c_{ij} x_{ij} \\ \text{s.t.} \quad & \sum_j^m x_{ij} \leq u_i \quad \text{for all } i = 1, \dots, n \\ & \sum_i^n x_{ij} \geq l_j \quad \text{for all } j = 1, \dots, m \\ & x_{ij} \geq 0 \end{aligned}$$

n factories $i = 1, \dots, n$ with supplies u_i

m warehouses $j = 1, \dots, m$ with minimum demands l_j

Transportation costs c_{ij} for one unit between factory i and warehouse j

General Linear Program

$$\max \quad c_1x_1 + c_2x_2 + \dots + c_nx_n$$

$$\text{s.t.} \quad a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

$$\vdots \quad \quad \quad \vdots \quad \quad \quad \vdots \quad \quad \quad \vdots$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

$$x_1 \geq 0, x_2 \geq 0, \dots, x_n \geq 0$$

The number of variables is n , the number of restrictions is m .

General Linear Program

Matrix and vector notation with

- n -dim cost vector c
- n -dim variable vector x
- m -dim “right-hand side vector” $b \geq \vec{0}$
- $(m \times n)$ -dim “constraint matrix” A

$$\begin{array}{ll} \max & c^T x \\ \text{s.t.} & Ax = b \\ & x \geq \vec{0} \text{ [often written as } x \in \mathbb{R}_{\geq 0}^n \text{]} \end{array}$$

This notation is also called the standard form of LPs.

Assumptions in LPs

- Proportionality
 - The contribution of a decision variable to the objective function is proportional to the value of the variable.
- Additivity / linearity
 - The total cost of food is equal to the sum of the individual food items.
- Divisibility
 - Each decision can take any possible value (real number), e.g., the daily amount of milk.
- Certainty of the data
 - The parameters of the model are known with certainty.
 - This assumption is often not fulfilled.
 - It is often difficult to get good data for estimates of parameters.
 - Sensitivity analysis can help.

Data types in optimization

- Continuous
 - includes real numbers ($x \in \mathbb{R}$)
 - possibly with non-negativity constraint “ $x \geq 0$ ”
 - e.g., how much money should be invested?
- Binary
 - restriction “ $x \in \{0, 1\}$ ”, yes/no values
 - e.g., whether an investment should be made or not
- Integer variable
 - restriction “ $x \in \mathbb{Z}$ ” or “ x integer”
 - e.g., number of workers per shift
- Finite Quantity
 - restriction “ $x \in S$ ”, where S is the set of values that x can assume
 - e.g., which one of 3 investments should be made

Rostering

An airline faces decreasing demand.

In order to reduce costs, the number of workers at the Munich site should be reduced while maintaining a satisfactory service level.

Linear programming can be used to specify the minimum number of workers that must be present per hour.

The costs of hiring a worker depend on the shift.

The airline wants to minimize the number of workers per shift while maintaining a certain service level.

Roster

Time period	Time frame					Minimum number of required workers
	Shift					
	1	2	3	4	5	
6 am to 8 am	x					48
8 am to 10 am	x	x				79
10 am to 12 am	x	x				65
12 am to 2 pm	x	x	x			87
2 pm to 4 pm		x	x			64
4 pm to 6 pm			x	x		73
6 pm to 8 pm			x	x		82
8 pm to 10 pm				x		43
10 pm to 12 pm				x	x	52
12 pm to 6 am					x	15

Rostering model

- Model parameters:
 - 5 shifts, where c_s is the cost for a worker in shift $s = 1, \dots, 5$
- Variables:
 - x_s : number of workers who must be employed in shift s
- Objective function:
 - minimize total costs
- Constraints / parameters:
 - 10 time periods $t = 1, \dots, 10$ with minimum number of required workers l_t
 - For each time period $t = 1, \dots, 10$, a subset $T_t \subseteq \{1, \dots, 5\}$ of shifts which fall into this period

Integer Linear Program:

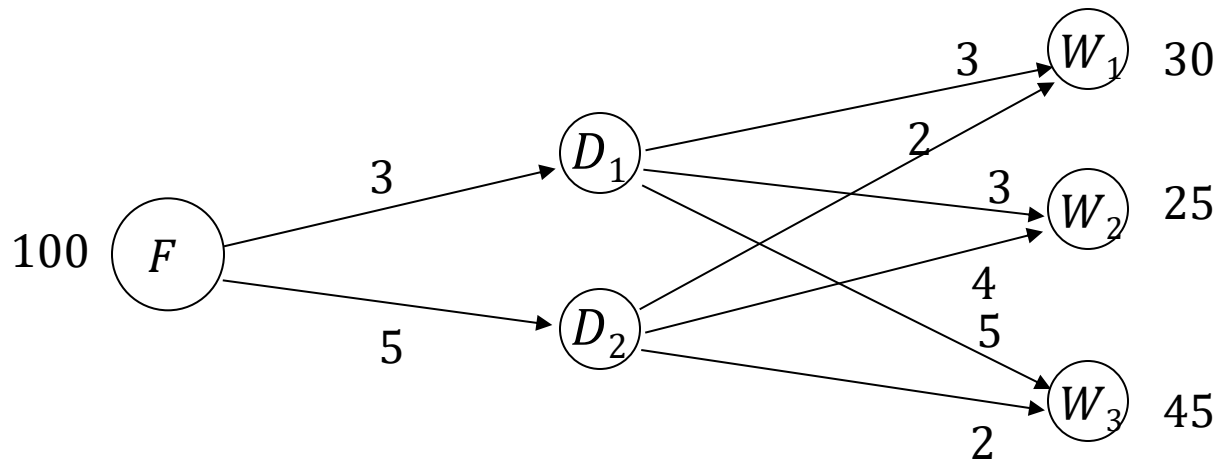
$$\begin{aligned} \min \quad & \sum_{s=1}^5 c_s x_s \\ \text{s.t.} \quad & \sum_{s \in T_t} x_s \geq l_t \quad \text{for } t = 1, \dots, 10 \\ & x_s \in \mathbb{Z}_{\geq 0} \quad \text{for } s = 1, \dots, 5 \end{aligned}$$

Transshipment problem

The weekly quantity produced in a factory equals the demand in all warehouses.

One can send any quantity on any route of the distribution network.

Assume that the distribution centers have capacity constraints.



Knapsack problem

A hiker can carry different useful/valuable items of different weights in his knapsack.

Which ones should he choose so that the maximum benefit is obtained at a given maximum weight?



Modeling the knapsack problem

- Model:
 - n items, $j = 1, \dots, n$ with value c_j and weight w_j
- Variables:
 - x_j for $j = 1, \dots, n$, if item j is picked
- Objective function:
 - maximize the values of the picked items
- Constraints:
 - the knapsack has a maximum weight W

Binary Linear Program:

$$\begin{array}{ll} \max & \sum_{j=1}^n c_j x_j \\ \text{s.t.} & \sum_{j=1}^n w_j x_j \leq W \\ & x_j \in \{0, 1\} \quad \text{for } j = 1, \dots, n \end{array}$$

Comparison LP vs. IP

Production planning problem with 200 products and 20 constraints

```
# Create variables
products = {}
for j in range(n):
    products[j] = solver.NumVar(0, solver.infinity(), f'Product{j}')
```

Solved in 21 iterations and 0.00 seconds

```
# Create variables
products = {}
for j in range(n):
    products[j] = solver.IntVar(0, solver.infinity(), f'Product{j}')
```

Explored 731868 nodes (2162223 simplex iterations) in 85.05 seconds

Data types: overview

- (Real-values) Linear Programming (LP) [**Lectures 2-7**]
 - fast solvable
 - solution space has more structure
- Integer Linear Programming (IP) [**Lectures 8-10**]
 - includes binary variables and variables from finite sets
 - allows more modeling possibilities
 - harder to solve, sometimes only approximately
 - **solution techniques often use continuous linear programming methods**
- For many integer problems even continuous variables are sufficient [**Lectures 11,12**]

Non-linear objective functions and constraints

Examples of non-linear functions:

- Polynomials of degree 2 or higher: $f(x, y, z) = x^2 + y^2 + z^2$
- Multiplication: $f(x, y, z) = x \cdot y$
- Exponential function: $f(x) = e^x$
- Maximum: $f(x, y, z) = \max\{x, y, z\}$
- Absolute value: $f(x) = |x|$
- ...

Even harder to solve, but:

- A large number of practical problems can be modeled as LP/IP
- Many nonlinear functions can be (approximately) linearized.

Assignment problem

An installation company has 3 projects to complete and 3 teams available.

Teams vary in how efficient they are at their work.

The overview shows how long each team is expected to take to complete each project:

<u>Team</u>	<u>Project</u>		
	<u>A</u>	<u>B</u>	<u>C</u>
Albert, Lara	28	30	18
Claudia, Michael	35	32	20
Doris, Tina	25	25	14

How to minimize the total number of hours?